MicroLA Class Hierarchy (C++20-C++26)

microla namespace

SquareMat *T, N*

Alias of Mat<T,N,N>

+same type and square-matrix helpers as Mat
+rotation_x(T)
+rotation_y(T)
+rotation_z(T)
+rotation_axis_angle(axis, angle)
+look_at(target, up) {N==3}
+euler_angles() {N==3}

Convenience alias, not a separate runtime type

alias of

VectorView *T, N*

+T& operator[](std::size_t)
+const T& operator[](std::size_t) const
+void set(Vec<T,N>)
+Vec<T,N> to_vec() const
+void fill(T)
+size_t size()
+size_t stride()

Non-owning subvector
Supports stride for columns/interleaved data

Utility Functions

«namespace» constants

+pi<T>()
+two_pi<T>()
+deg_to_rad<T>()
+rad_to_deg<T>()
+epsilon<T>()

C++20+ variable templates
Callable constant proxies

«namespace» memory_info

+matrix_size_bytes<T,R,C>()
+vector_size_bytes<T,N>()
+quaternion_size_bytes<T>()
+validate_matrix_stack_size<>()

constexpr memory queries
Embedded-friendly

«functions» safe_math

+safe_divide<T>(num, den, default_val)
+safe_reciprocal<T>(x, default_val)
+saturating_add/sub/mul<T>(a, b)
+safe_sqrt<T>(value)
+safe_acos<T>(value)
+safe_asin<T>(value)
+is_finite<T>(x), is_nan<T>(x)

Safety-critical operations
All default to float

«namespace» fast

+sin<T>(x), cos<T>(x), tan<T>(x)
+atan2<T>(y, x)
+rsqrt<T>(x), sqrt<T>(x)
+exp<T>(x), ln<T>(x), pow<T>(b, e)
+lerp<T>(a, b, t)
+smoothstep<T>(edge0, edge1, x)

Operation-specific approximations and domains
Large-angle sine/cosine use standard math

«functions» numerical

+approx_equal<T>(a, b, rel_tol, abs_tol)
+kahan_sum<T, Iterator>(begin, end)
+hypot<T>(a, b), hypot3<T>(a, b, c)
+log1p<T>(x), expm1<T>(x)
+stable_dot_product<T>(a, b, n)
+condition_number_2x2<T>(...)
+precision_loss_bits<T>(...)
+ulp_distance<T>(a, b)
+horner_eval<T>(coeffs, n, x)

Numerically stable algorithms

«structs» resource_checks

+VecLayoutCheck<T, N>
+MatSizeInfo<T, R, C>
+QuaternionSizeInfo<T>

Compile-time size/layout checks
Standard-layout/trivial-copy checks
DMA requirements remain target-specific

Features:
• Row-major storage
  • Submatrices use non-owning MatrixView
  • Square determinant/caller-owned inverse
  • Optional NEON 4x4 and CMSIS products
• constexpr compatible
  • C++20-C++26 optimizations

Mat *T, R, C*

+alignas(MICROLA_DATA_ALIGNMENT) T data[R*C]

+Mat()
+Mat(initializer_list)
+static Mat identity() {R==C}
+static Mat zero()
+T& operator()(std::size_t, std::size_t)
+Mat operator+(Mat)
+Mat operator-(Mat)
+Mat operator*(Mat)
+Mat operator*(T)
+Mat transpose()
+T trace() {R==C}
+T determinant() {R==C}
+bool inverse(Mat& out) {R==C}
+bool inverse_checked(Mat& out, T threshold) {R==C}
+T frobenius_norm()

Row-major storage
Contiguous row-streaming float multiplication
C++20-C++26 adaptive

to_matrix()

from rotation matrix

Quaternion *T*

+alignas(MICROLA_DATA_ALIGNMENT) T data[4]
+// [x, y, z, w] layout

+Quaternion()
+Quaternion(w, x, y, z)
+Quaternion(axis, angle)
+Quaternion(Mat<T,3,3>)
+Quaternion operator*(Quaternion)
+Vec<T,3> rotate(Vec<T,3>)
+Quaternion conjugate()
+Quaternion inverse()
+T norm()
+void normalize()
+Mat<T,3,3> to_matrix()
+Vec<T,3> to_euler()
+Quaternion lerp(Quaternion, T)
+Quaternion slerp(Quaternion, T)

Selected optional backend paths; RVV experimental
Optimized Vec3 integration
Template default: T = float

Features:
• [x,y,z,w] memory layout
• Vec<T,3> integration
• SLERP interpolation
  • Selected NEON/CMSIS paths; measure on target
  • Sign/scale-invariant shortest angle_to
• constexpr compatible

matrix-vector products

covariance matrices

rotate vector

covariance matrices

Vec *T, N*

+alignas(MICROLA_DATA_ALIGNMENT) T data[N]

+Vec()
+Vec(T...)
+T& operator[](std::size_t)
+Vec operator+(Vec)
+Vec operator-(Vec)
+Vec operator*(T)
+Vec operator/(T)
+T dot(Vec)
+Vec cross(Vec) {N==3}
+T length()
+T length_squared()
+Vec normalized()
+Vec lerp(Vec, T)

SIMD: AVX/NEON/CMSIS/RISC-V
C++20-C++26 adaptive

Features:
• N-dimensional vectors
• Component access (x,y,z,w)
• SIMD: AVX/NEON/CMSIS/RISC-V for float 2/3/4
• constexpr compatible
• Zero allocation

state vector

state vector

KalmanFilter *T, StateDim, MeasDim*

-Vec<T, StateDim> x
-Mat<T, StateDim, StateDim> P
-Mat<T, StateDim, StateDim> F
-Mat<T, MeasDim, StateDim> H
-Mat<T, StateDim, StateDim> Q
-Mat<T, MeasDim, MeasDim> R

+void predict()
+void predict(control, B)
+bool update(measurement)
+MeasVec compute_innovation(z)
+MeasMat compute_innovation_covariance()
+T compute_nis(z)
+void set_state_transition(F)
+void set_measurement_matrix(H)
+void set_process_noise(Q)
+void set_measurement_noise(R)

Joseph form covariance update
Innovation & NIS computation
Template defaults to float

ExtendedKalmanFilter *T, StateDim, MeasDim*

-Vec<T, StateDim> x
-Mat<T, StateDim, StateDim> P
-Mat<T, StateDim, StateDim> Q
-Mat<T, MeasDim, MeasDim> R
-StateTransitionFunc f
-StateJacobianFunc F_jacobian
-MeasurementFunc h
-MeasurementJacobianFunc H_jacobian

+void predict(dt)
+bool update(measurement)
+bool update_joseph(measurement)
+void set_state_transition(f, F_jac)
+void set_measurement_model(h, H_jac)

Nonlinear system support
Jacobian linearization
Template defaults to float